

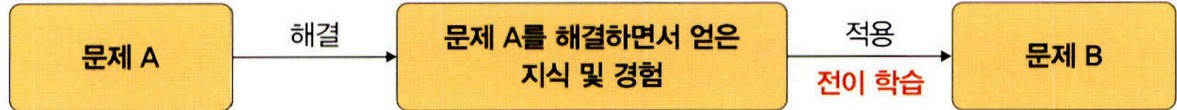
4주차 : 딥러닝 파이토치 교과서 5장

Part 2(5.3~)

5.3 : 전이 학습

- 전이 학습 : 아주 큰 데이터셋(ex. ImageNet)을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것
 - 일반적으로 CNN 기반 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요하나, 충분히 큰 데이터셋을 얻는 것은 쉽지 않음 → 이러한 어려움을 해결한 것이 전이 학습
⇒ 결과적으로 비교적 적은 수의 데이터를 가지고도 우리가 원하는 과제를 해결할 수 있음
 - 아주 큰 데이터셋을 사용하여 훈련된 모델 = 사전 훈련된 모델(네트워크)

▼ 그림 5-29 전이 학습



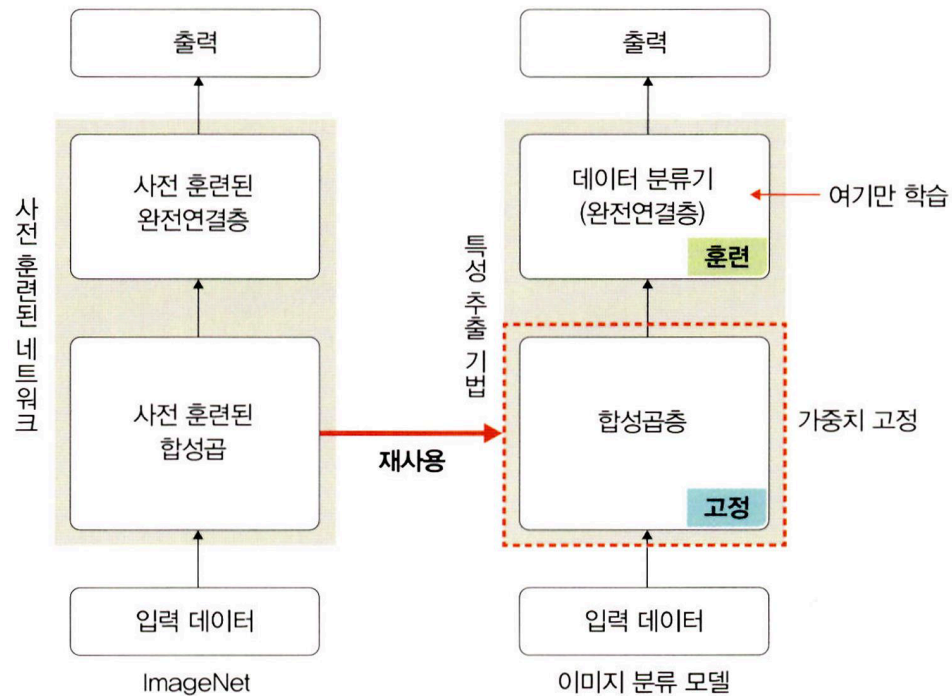
- 전이 학습을 위한 방법
 - a. 특성 추출
 - b. 미세 조정 기법

5.3.1: 특성 추출 기법

- 특성 추출(feature extractor) : ImageNet 데이터셋으로 사전 훈련된 모델을 가져옴
→ 마지막 완전연결층 부분만 새로 만들
⇒ 마지막 완전연결층(이미지의 카테고리 결정)만 학습하고 나머지 계층들은 학습되지 않도록 함
- 특성 추출의 구성

- 합성곱층 : 합성곱층과 풀링층으로 구성
 - 사전 훈련된 네트워크의 합성곱층에 데이터 통과
- 데이터 분류기(완전연결층) : 추출된 특성 입력받아 최종적으로 이미지에 대한 클래스 분류
 - 위의 출력을 데이터 분류기에서 훈련시킴

♥ 그림 5-30 특성 추출 기법



```
# 코드 5-12 : 라이브러리 호출
# pip install opencv-python
```

```
import os
import time
import copy
import glob
import cv2
import shutil
```

```
import torch
import torchvision # 컴퓨터 비전 용도의 패키지
```

```

import torchvision.transforms as transforms # 데이터 전처리를 위
해 사용되는 패키지
import torchvision.models as models # 다양한 파이토치 네트워크를
사용할 수 있도록 도와주는 패키지
import torch.nn as nn
import torch.optim as optim

import matplotlib.pyplot as plt

```

코드 5-13 : 이미지 데이터 전처리 방법 정의

```

data_path = 'catanddog/train/'

# torchvision.transform은 이미지 데이터를 변환하여 모델(네트워크)의
입력으로 사용할 수 있게 변환해 줌
transform = transforms.Compose(
    [
        transforms.Resize([256, 256]), # 이미지 크
기 조정
        transforms.RandomResizedCrop(224), # 이미
지를 랜덤한 크기 및 비율로 자름
        transforms.RandomHorizontalFlip(), # 이미
지를 랜덤하게 수평으로 뒤집음
        transforms.ToTensor(), # 이미지 데이터를 텐
서로 변환
    ])
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset, # 데이터셋 지정
    batch_size=32, # 한번에 불러올 데이터양 결정
    num_workers=8, # 데이터를 불러올 때 하위 프로세스를 몇개 사용할
건지

```

```

        shuffle=True # 데이터를 무작위로 섞어서 불러옴
    )

print(len(train_dataset))

```

- Resize : 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정
- RandomResizedCrop : 데이터 확장 용도 → 이미지를 랜덤한 비율로 자른 후 데이터 크기 조정
 - 데이터 확장 : 가진 데이터의 양과 다양성을 인위적으로 늘려서 AI 모델이 더 똑똑하고 유연하게 학습하도록 돕는 기법
 - 정리하자면, **Resize** 는 모델에 입력을 넣기 위한 최소한의 규칙 맞추기라면, **RandomResizedCrop** 은 모델이 다양한 상황을 경험할 수 있도록 시험문제를 매번 조금씩 변형해서 내는 '데이터 확장' 기법입니다.

코드 5-14 : 학습에 사용될 이미지 출력

```

import numpy as np
samples, labels = iter(train_loader).next() # 반복자 - for 구문
과 같은 효과
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24):
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()])
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) # 행과
열 바꿈 -> 행렬의 차원 바꿈
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

```

- 행렬의 차원을 변경하는 이유 : 행렬의 내적 연산
 - 내적이 가능하기 위해서는 두 행렬 각각의 행과 열이 같아야 하기 때문

코드 5-15 : 사전 훈련된 모델 내려받기

```

resnet18 = models.resnet18(pretrained=True) # 사전 학습된 가중치

```


를 사용하겠다는 의미의 옵션

- ResNet18 : 50개의 계층으로 구성된 합성곱 신경망
→ ImageNet 데이터베이스의 100만개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공함

코드 5-16 : 사전 훈련된 모델의 파라미터 학습 유무 지정

```
def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False # 파라미터들에 대한 변화를  
            # 계산할 필요가 없음을 나타냄 (=모델의 일부를 고정하고 학습)  
  
set_parameter_requires_grad(resnet18)
```

- 모델의 일부를 고정하고 나머지를 학습하고자 할 때 requires_grad = False로 설정
 - 모델의 일부 → 합성곱층, 풀링층

코드 5-17 : ResNet18에 완전연결층 추가

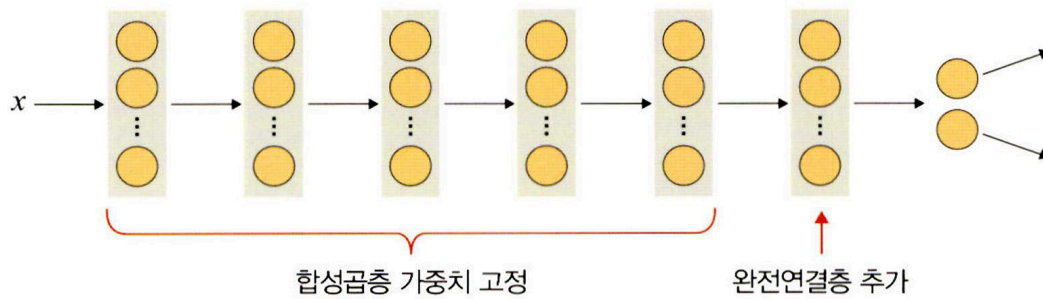
```
resnet18.fc = nn.Linear(512, 2) # 이때의 2 -> 클래스가 두개라는  
# 의미
```

코드 5-18 : 모델의 파라미터 값 확인

```
for name, param in resnet18.named_parameters(): # model.named_  
_parameters() : 모델에 접근하여 파라미터 값들을 가져올 때 사용  
    if param.requires_grad:  
        print(name, param.data)
```

- 추가된 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용됨

▼ 그림 5-36 ResNet18에 완전연결층 추가



코드 5-19 : 모델 객체 생성 및 손실 함수 정의

```
model = models.resnet18(pretrained = True) # 모델의 객체 생성

for param in model.parameters(): # 모델의 합성곱층 가중치 고정
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters(): # 완전연결층은 학습
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실 함수 정의
print(model)
```

코드 5-20 : 모델 학습을 위한 함수 생성

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13, is_train=True):
    since = time.time()
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
```

```

print('Epoch {}/{}'.format(epoch, num_epochs - 1))
print('-' * 10)

running_loss = 0.0
running_corrects = 0

for inputs, labels in dataloaders: # 데이터로더에 전달된
데이터만큼 반복
    inputs = inputs.to(device)
    labels = labels.to(device)

    model.to(device)
    optimizer.zero_grad() # 기울기를 0으로 설정
    outputs = model(inputs) # 순전파 학습
    loss = criterion(outputs, labels)
    _, preds = torch.max(outputs, 1)
    loss.backward() # 역전파 학습
    optimizer.step()

    running_loss += loss.item() * inputs.size(0)
    running_corrects += torch.sum(preds == labels.data)

a)

epoch_loss = running_loss / len(dataloaders.dataset)
# 평균 오차 계산
epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 평균 정확도 계산

print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

if epoch_acc > best_acc:
    best_acc = epoch_acc

acc_history.append(epoch_acc.item())
loss_history.append(epoch_loss)

```

```

        torch.save(model.state_dict(), os.path.join('catanddog/', '{0:0=2d}.pth'.format(epoch)))
        print()

```

```

        time_elapsed = time.time() - since # 실행 시간(학습 시간)을
계산
        print('Training complete in {:.0f}m {:.0f}s'.format(time_
elapsed // 60, time_elapsed % 60))
        print('Best Acc: {:.4f}'.format(best_acc))
        return acc_history, loss_history

```

코드 5-21 : 파라미터 학습 결과를 옵티마이저에 전달

```

params_to_update = []
for name,param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과를 저
장
        print("\t",name)

```

```

optimizer = optim.Adam(params_to_update) # 학습 결과를 옵티마이
저에 전달

```

코드 5-22 : 모델 학습 (train data)

```

device = torch.device("cuda" if torch.cuda.is_available() els
e "cpu")
criterion = nn.CrossEntropyLoss() # 손실 함수 지정
train_acc_hist, train_loss_hist = train_model(resnet18, train
_loader, criterion, optimizer, device)

```

코드 5-23 : 테스트 데이터 호출 및 전처리

```

test_path = 'catanddog/validation/'

```

```

transform = transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
    ])
test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32,
    num_workers=1,
    shuffle=True
)

print(len(test_dataset))

```

코드 5-24 : 테스트 데이터 평가 함수 생성

```

def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('catanddog/' + '*.pth')
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))

```

```

model.eval()
model.to(device)
running_corrects = 0

for inputs, labels in dataloaders: # 테스트 반복
    inputs = inputs.to(device)
    labels = labels.to(device)

    with torch.no_grad(): # autograd를 사용하지 않겠다는
        outputs = model(inputs) # 데이터를 모델에 적용한
        # 결과를 outputs에 저장

        _, preds = torch.max(outputs.data, 1)
        preds[preds >= 0.5] = 1 # torch.max로 출력된 값이
        # 0.5보다 크면 올바르게 예측
        preds[preds < 0.5] = 0 # 작으면 틀리게 예측
        running_corrects += preds.eq(labels).int().sum()

    epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 테스트 데이터의 정확도 계산
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc

    acc_history.append(epoch_acc.item())
    print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history

```

```

# 코드 5-24 수정
# 현재 코드의 흐름은 아래와 같다.
model
↓
outputs
↓
torch.max(outputs, 1)
↓
최댓값의 인덱스
↓
preds = 0 또는 1
↓
labels와 비교
↓
정확도 계산

# 코드 수정 필요 (현재)
preds[preds >= 0.5] = 1
preds[preds < 0.5] = 0

# 코드 수정
outputs = model(inputs)

_, preds = torch.max(outputs, 1)

running_corrects += (preds == labels).sum()

# 수정된 코드의 흐름
torch.max()
↓
preds = 예측한 클래스 번호
↓
preds == labels ← 예측이 맞았는지 비교

```

↓
`sum()` ← 맞힌 개수 계산

- (GPT) 현재 코드에서는 0.5 기준으로 `preds` 를 다시 변환하는 과정이 결과에는 영향을 주지 않지만, 이미 클래스 인덱스가 결정된 값을 불필요하게 다시 처리하므로 제거하는 것이 적절하다.

코드 5-25 : 테스트 데이터를 평가 함수에 적용

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

코드 5-26 : 훈련과 테스트 데이터의 정확도를 그래프로 확인

```
plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```

코드 5-27 : 훈련 데이터의 오차에 대한 그래프 확인

```
plt.plot(train_loss_hist)
plt.show()
```

- 훈련과 테스트 데이터 모두 에포크가 진행될수록 정확도가 높아지는 양상을 보이고 있음
- 에포크가 진행될수록 오차가 낮아지고 있다 → 학습이 잘 되었다고 할 수 있음

코드 5-28 : 예측 이미지 출력을 위한 전처리 함수

```
def im_convert(tensor):
    image=tensor.clone().detach().numpy() # 기존 텐서의 내용을
    복사한 텐서를 생성하겠다는 의미
    image=image.transpose(1,2,0)
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5)))
    image=image.clip(0,1) # 입력 값이 주어진 범위를 벗어날 때 입력
```


값을 특정 범위로 제한
`return image`

- `detach()` : 기존 텐서에서 기울기가 전파되지 않는 텐서

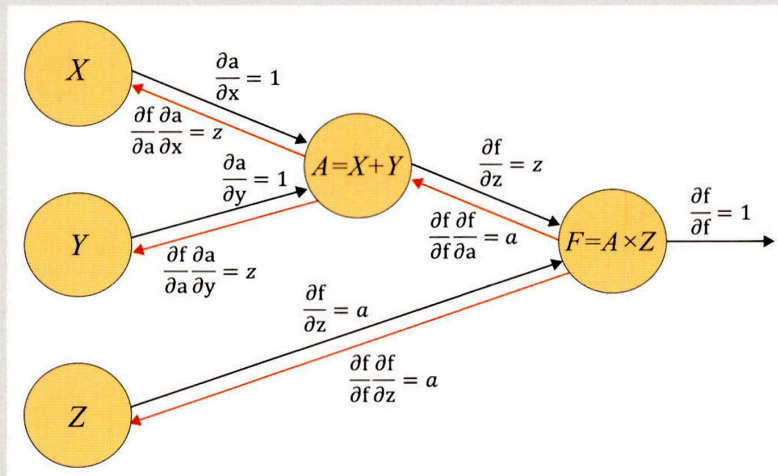
▼ 표 5-2 `tensor.clone()`, `tensor.detach()`, `tensor.clone().detach()`의 비교

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

Note ≡ 계산 그래프

계산 그래프(computational graph)는 이미 2장에서 살펴보았지만 리마인드를 위해 다시 살펴보겠습니다. 계산 그래프란 계산 과정을 그래프로 나타낸 것입니다. 그래프는 여러 개의 노드(node)와 그 노드들을 연결하는 선인 에지(edge)로 구성됩니다.

▼ 그림 5-39 계산 그래프



계산 그래프를 사용하는 이유는 두 가지입니다.

- 국소적 계산이 가능합니다. 국소적 계산이 가능하다는 의미는 그림 5-39에서 Z 값이 변경되었다면 X , Y 계산 결과를 그대로 유지한 채로 바뀐 Z 의 연산이 필요한 $F = A \times Z$ 만 계산하면 됩니다.
- 역전파를 통한 미분 계산이 편리합니다. 그림 5-39의 주황색 선이 역전파를 구하는 과정을 보여 주는데 연쇄 법칙(chain rule)을 이용하여 빠르고 간편하게 미분을 계산할 수 있습니다.

코드 5-29 : 개와 고양이 예측 결과 출력

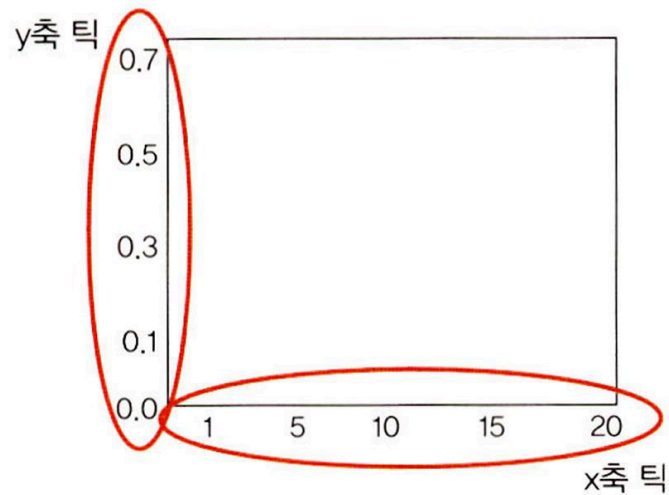
```
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 레이블

dataiter=iter(test_loader)
images,labels=dataiter.next() # 테스트 데이터셋에서 이미지와 레이블을 분리하여 가져옴
output=model(images)
_,preds=torch.max(output,1)

fig=plt.figure(figsize=(25,4))
for idx in np.arange(20):
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])
    plt.imshow(im_convert(images[idx])) # 이미지 출력을 위해 5-28에서 정의한 im_convert 함수를 적용
    a.set_title(classes[labels[i].item()]) # preds[idx].item() 값이 classes로 정의된 '0'과 '1'중 어떤 값을 갖는지 판별하겠다는 의미
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()])),color=("green" if preds[idx]==labels[idx] else "red"))
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- add_subplot : 한 화면에 여러개의 이미지를 담기 위해 사용
 - add_subplot(2, 10, idx+1, xticks=[],yticks=[]))
 - 첫 번째 파라미터 : 행의 수 → 이미지를 두줄로 출력
 - 두 번째 파라미터 : 열의 수 → 한 줄에 열개의 이미지 출력
 - 세 번째 파라미터 : 인덱스 → 순차적으로 이미지 출력
 - 네 번째 파라미터 : 틱을 삭제하겠다는 의미

♥ 그림 5-41 x축, y축 틱



5.3.2 : 미세 조정 기법

- 미세 조정 기법 (fine-tuning) : 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식 → 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것
 - 파라미터 업데이트 과정에서 큰 변화를 주면 과적합 문제가 발생할 수 있기 때문에 정교하고 미세한 파라미터 업데이트가 필요하다.
 - 훈련시키려는 데이터셋의 크기와 사전 훈련된 모델에 따라 전략을 세울 수 있다.

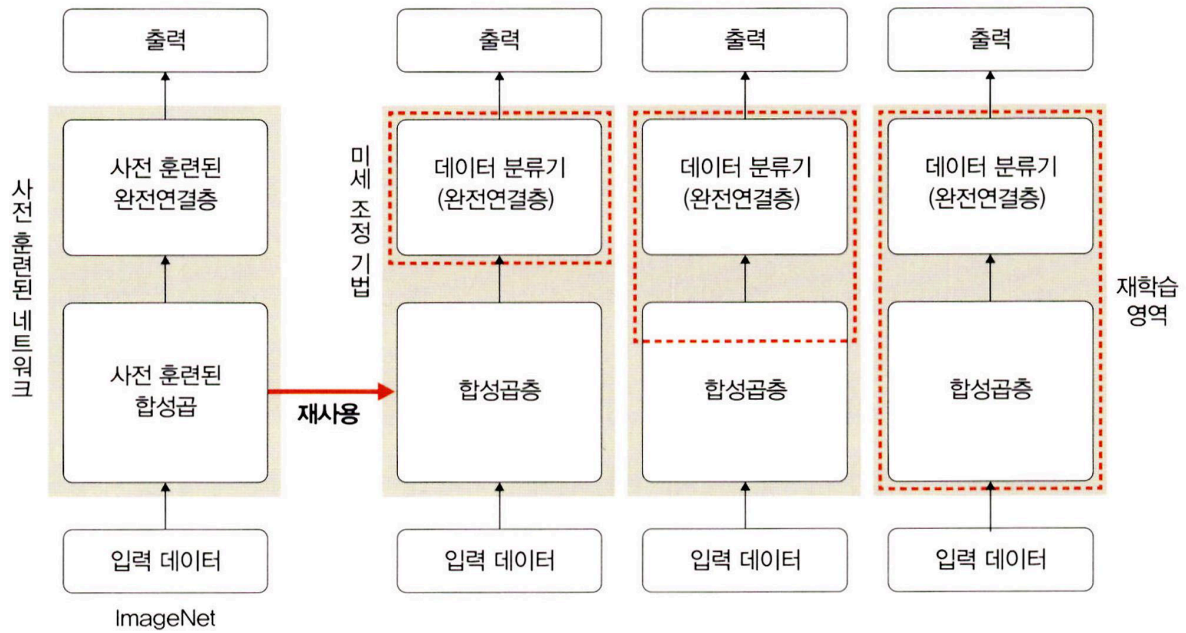
4가지 시나리오 총정리

이 개념은 전이 학습(Transfer Learning)의 표준 매트릭스로 정리해 두시면 이해가 훨씬 쉬워집니다.

구분	유사성 높음 (기존 모델과 비슷)	유사성 낮음 (기존 모델과 완전히 다름)
데이터 적음	특성 추출 (Feature Extraction) (과적합 방지를 위해 백본 동결, FC만 학습)	상단 일부만 미세 조정 (데이터가 적어 전체 학습 시 과적합 위험)
데이터 많음	약한 미세 조정 (Fine-Tuning) (사전 학습 지식이 유용하므로 가볍게 다듬음)	모델 전체 재학습 (Full Re-training) (데이터가 넉넉하므로 과적합 걱정 없이 내 데이터에 최적화)

- 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우 : 모델 전체를 재학습
→ 데이터셋 크기가 크기 때문에 재학습시키는 것이 좋은 전략이다.
- 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우 : 합성곱층의 뒷부분, 데이터 분류기 학습
→ 데이터셋이 유사하므로 일부만 새로 학습하더라도 좋은 성능을 낼 수 있다.
- 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우 : 합성곱층의 일부분, 데이터 분류기 학습
→ 데이터가 적기 때문에 일부 계층에 미세 조정 기법을 적용한다고 해도 효과가 없을 수 있음
⇒ 합성곱층 중 어디까지 새로 학습시켜야 할지 적당히 설정해야 함
- 데이터셋이 작고 사전 훈련된 모델 과 유사성이 클 경우 : 데이터 분류기만 학습
→ 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 과적합 발생할 수 있음
⇒ 최종 데이터 분류기인 완전연결층에 대해서만 미세 조정 기법 적용

▼ 그림 5-44 미세 조정 기법



5. 4 : 설명 가능한 CNN

- 설명 가능한 CNN (explainable CNN) : 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시
 - CNN은 블랙박스과 같아서 내부에서 어떻게 동작하는지 설명하기 어려움
 - CNN으로 얻은 결과를 신뢰하기 어려움
 - ⇒ 이를 해결하기 위해선 CNN 처리 과정을 시각화 해야 함 : 각 계층부터 최종 분류까지 입력된 이미지에서 특성이 어떻게 추출되고 학습하는지를 시각적으로 설명할 수 있어야 결과에 대한 신뢰성을 얻을 수 있다.

▼ 그림 5-45 CNN의 블랙박스



5. 4. 1 : 특성 맵 시각화

- 특성 맵 (feature map) : 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과

- 특정 입력 이미지에 대한 특성 맵을 시각화
→ 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것

```
# 코드 5-30 : 필요한 라이브러리 호출
# pip install pillow : Python Image Library 설치

import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
# 코드 5-31 : 설명 가능한 네트워크 생성

class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), # 기존의 데이터를 연산의 결과값으로 대체
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding = 1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
```



```

nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(64, 128, kernel_size=3, padding = 1, bi
as=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(128, 128, kernel_size=3, padding = 1, b
ias=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(128, 256, kernel_size=3, padding = 1, b
ias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding = 1, b
ias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding = 1, b
ias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(256, 512, kernel_size=3, padding = 1, b
ias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),

```

```

        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding = 1, b
ias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),
    )
    self.classifier = nn.Sequential(
        nn.Linear(512, 512, bias=False),
        nn.Dropout(0.5),
        nn.BatchNorm1d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

    def forward(self, x):
        x = self.features(x)

```



```
x = x.view(-1, 512)
x = self.classifier(x)
return F.log_softmax(x) # 로그 소프트맥스 활용
```

- 로그 소프트맥스 : 신경망 말단의 결괏값들을 확률 개념으로 해석하기 위해 소프트맥스 함수의 결과에 log 값을 취한 연산
 - 소프트맥스 대신 로그 소프트맥스를 사용하는 이유 : 소프트맥스는 기울기 소멸 문제에 취약하기 때문

코드 5-32 : 모델 객체화

```
model=XAI() # 객체 생성
model.cpu() # 모델을 장치에 할당 - 모델에 입력되는 이미지를 넘파이로 받아오는 부분때문에 CPU를 사용하도록 지정하였습니다
model.eval() # 테스트 데이터에 대한 모델 평가 용도로 사용
```

코드 5-33 : 특성 맵을 확인하기 위한 클래스 정의

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        output = output
        #self.features = output.to(device).detach().numpy()
        self.features = output.detach().numpy()
    def remove(self):
        self.hook.remove()
```

코드 5-34 : 이미지 호출

```
from google.colab import files # 데이터 불러오기
```

```

file_uploaded=files.upload() # chap05/data/cat.jpg 데이터 불러오기

img=cv2.imread("cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR) # 이미지 크기 변경
img = ToTensor()(img).unsqueeze(0) # 1차원 데이터를 생성하는 함수

print(img.shape)

```

- 이미지 크기를 변경할 경우 이미지의 픽셀을 추정해서 값을 할당해야 함
→ 추정해야 하는 픽셀을 보간법(interpolation)을 이용하여 값을 할당

코드 5-35 : Conv2d 특성 맵 확인

```

result = LayerActivations(model.features, 0) # 0번째 Conv2d 특성 맵 확인
model(img)
activations = result.features

```

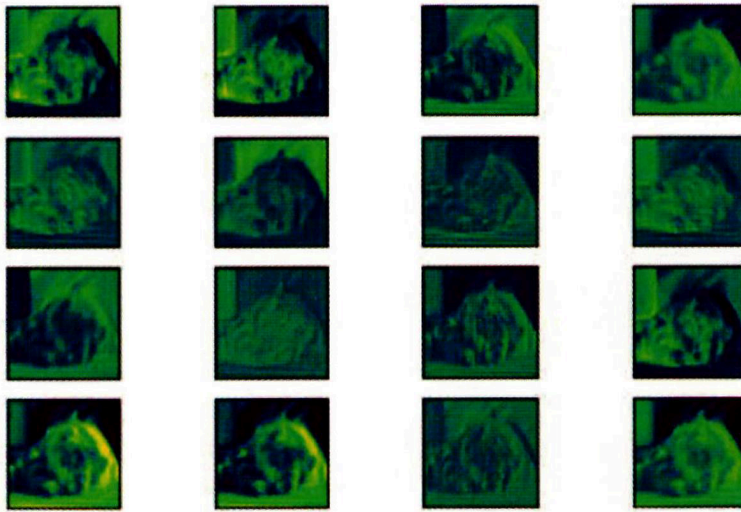
코드 5-36 : 특성 맵 확인

```

fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()

```

▼ 그림 5-47 첫 번째 계층에서 특성 맵



코드 5-37 : 20번째 계층에 대한 특성 맵

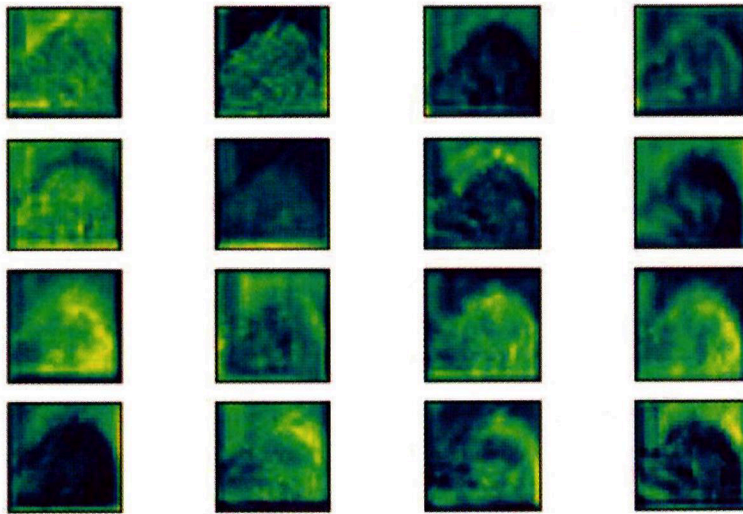
```
result = LayerActivations(model.features, 20) # 20번째 Conv2d  
특성 맵 확인
```

```
model(img)  
activations = result.features
```

코드 5-38 : 특성 맵 확인

```
fig, axes = plt.subplots(4,4)  
fig = plt.figure(figsize=(12, 8))  
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=  
0.05, wspace=0.05)  
for row in range(4):  
    for column in range(4):  
        axis = axes[row][column]  
        axis.get_xaxis().set_ticks([])  
        axis.get_yaxis().set_ticks([])  
        axis.imshow(activations[0][row*10+column])  
plt.show()
```

▼ 그림 5-48 20번째 계층에서 특성 맵



코드 5-39 : 40번째 계층에 대한 특성 맵

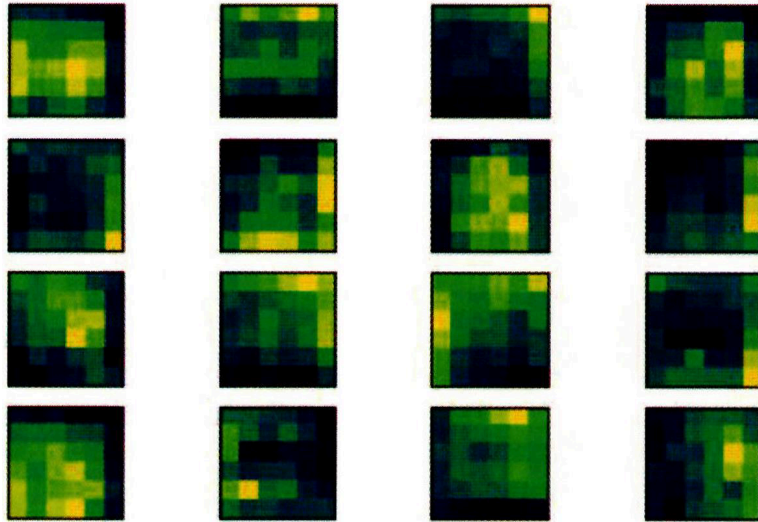
```
result = LayerActivations(model.features, 40) # 40번째 Conv2d  
특성 맵 확인
```

```
model(img)  
activations = result.features
```

코드 5-40 : 특성 맵 확인

```
fig, axes = plt.subplots(4,4)  
fig = plt.figure(figsize=(12, 8))  
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=  
0.05, wspace=0.05)  
for row in range(4):  
    for column in range(4):  
        axis = axes[row][column]  
        axis.get_xaxis().set_ticks([])  
        axis.get_yaxis().set_ticks([])  
        axis.imshow(activations[0][row*10+column])  
plt.show()
```

▼ 그림 5-49 40번째 계층에서 특성 맵



- 원래 입력 이미지에 대한 형태를 찾아볼 수 없음
→ 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징들만 전달되는 것을 확인할 수 있음

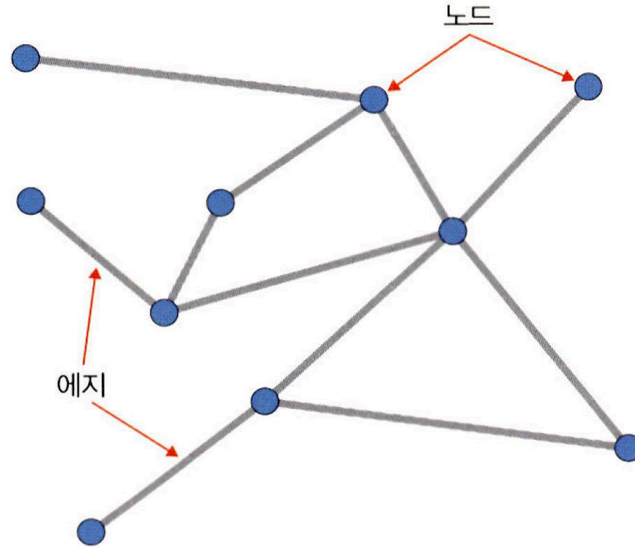
5.5 : 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(graph convolutional network) : 그래프 데이터를 위한 신경망

5.5.1 : 그래프란

- 그래프 : 방향성이 있거나(directed) 없는(undirected) 에지로 연결된 노드의 집합

▼ 그림 5-50 그래프

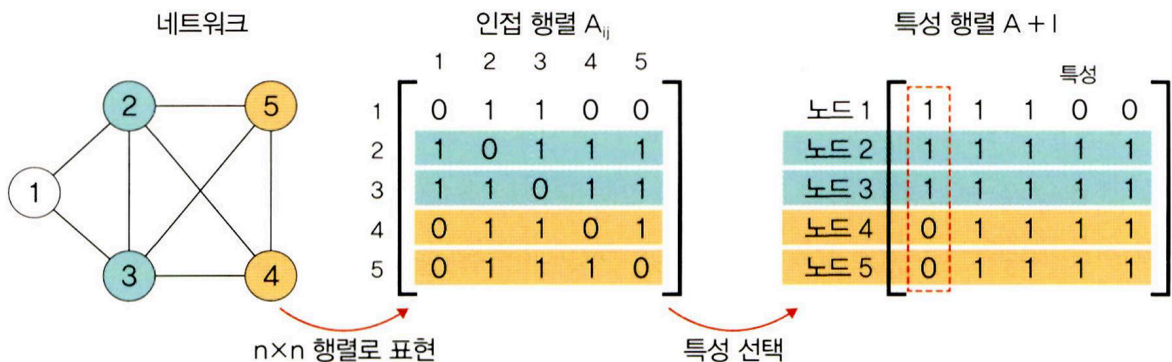


- 노드(node, vertex) : 파란색 원 → 원소들을 의미함
- 에지(edge) : 두 노드를 연결한 선 → 결합 방법을 의미함 (single, double, triple...)

5.5.2 : 그래프 신경망

- 그래프 신경망 (Graph Neural Network, GNN) : 그래프 구조에서 사용하는 신경망

▼ 그림 5-51 특성 행렬^b

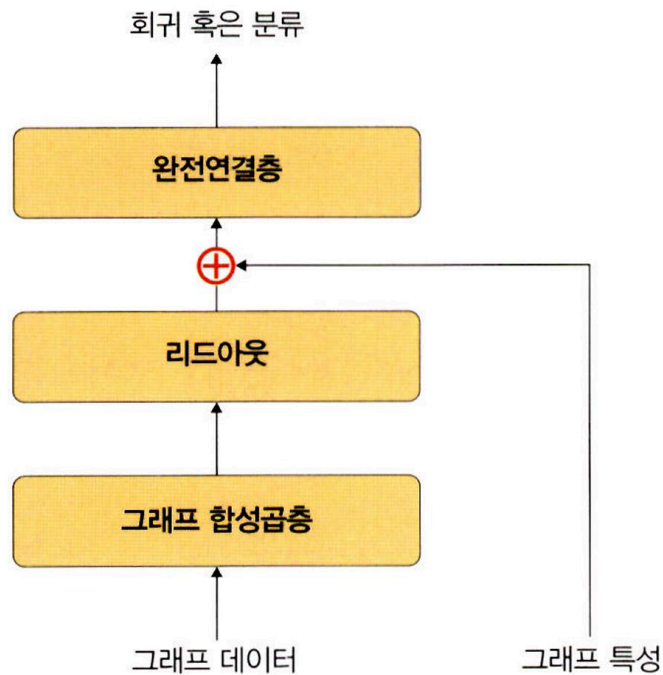


- 1단계 : 인접 행렬 (adjacency matrix) : 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정
- 2단계 : 특성 행렬 (feature matrix) : 각 입력 데이터에서 이용할 특성 선택 → 그래프 특성 추출

5.5.3 : 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크 (Graph Convolutional Network, GCN) : 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

▼ 그림 5-52 그래프 합성곱 네트워크



- 리드아웃 (readout) : 특성 행렬을 하나의 벡터로 변환하는 함수
→ 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성
- 그래프 합성곱층 (graph convolutional layer) : 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환됨 → 딥러닝 알고리즘 적용 가능 (기존 형태로는 적용 불가)